

CS 402: Web Architecture & Systems Engineering

An Exhaustive Rigorous Syllabus on Web Protocols, Distributed Client-Server Systems, State Management & Cloud Native Web Engineering

Course Level:	Undergraduate (Core CS / Advanced Systems Track)	Term / Year:	Academic Year 2026–2027
Prerequisites:	Computer Networks, Operating Systems, Database Management	Format:	4 Credit Hours (3h Lecture + 3h Engineering Lab / week)

1. COURSE PHILOSOPHY & ENGINEERING FRAMEWORK

Web Engineering focuses on the application of systematic, disciplined, and quantifiable software engineering principles to the development, deployment, and maintenance of high-performance, fault-tolerant web applications. Rather than focusing merely on UI frameworks or superficial web design, this course explores the underlying network dynamics, protocol abstractions, browser engine internals, server-side scaling topologies, state management mechanisms, and web security models.

Students will analyze the mechanics of the HTTP/1.x, HTTP/2, and HTTP/3 (QUIC) protocols, examine client-side rendering engine lifecycles (DOM parsing, style calculation, layout, paint, composite), design production-grade RESTful and GraphQL API contracts, and construct distributed microservices architectures capable of handling asynchronous event streams, caching strategies, and global edge deployments.

Core Engineering Principle: *"Design for scalability, resilience, and determinism across unreliable networks."* Students evaluate system trade-offs between latency, throughput, client-side execution overhead, state synchronization, and end-to-end security guarantees.

2. COMPREHENSIVE 16-WEEK CURRICULUM

WEEK	MODULE TOPIC	RIGOROUS CONTENT & THEORETICAL FOUNDATIONS
Week 1	The Web Infrastructure & Architecture CORE	Client-Server model evolution, Domain Name System (DNS) resolution mechanics, Anycast routing, Content Delivery Networks (CDNs), Reverse proxies (Nginx, HAProxy), TLS/SSL handshake protocol internals.
Week 2	Transport & Application Web Protocols CORE	HTTP/1.1 (Keep-Alive, Pipelining, Head-of-Line Blocking); HTTP/2 (Multiplexing, Binary Framing, HPACK Header Compression, Server Push); HTTP/3 over QUIC (UDP foundation, 0-RTT handshakes, connection migration).
Week 3	Browser Engine Architecture & Rendering Mechanics CORE	Browser processing pipeline: Network layer, HTML/CSS Parsing, DOM & CSSOM construction, Render Tree, Layout (Reflow), Paint, GPU Compositing. Event loop execution model (Microtasks vs Macrotasks).
Week 4	Semantic Markup, Accessibility & CSS Engine CORE	DOM Spec, Shadow DOM, Accessibility Tree (a11y), ARIA semantics. CSS layout engines (Formatting contexts, Stacking contexts), BEM methodology, Critical Rendering Path optimization, CSS Containment.

WEEK	MODULE TOPIC	RIGOROUS CONTENT & THEORETICAL FOUNDATIONS
Week 5	Client-Side Runtime & Asynchronous JavaScript ADVANCED	ECMAScript execution contexts, Memory management & Garbage Collection (Mark-and-Sweep), Closures, Prototypes, Promises, <code>async/await</code> , Web Workers, SharedArrayBuffer, and Atomics.
Week 6	Modern Frontend Framework Architecture ADVANCED	Virtual DOM reconciliation algorithms (Fiber architecture) vs Fine-Grained Reactivity (Signals), Component Lifecycles, Client-Side Routing mechanics, Single Page Application (SPA) state synchronization.
Week 7	Midterm Examination & Systems Audit EVAL	Theoretical exam and system design diagnostic covering protocol packet flows, critical rendering path analysis, and API design principles.
Week 8	Server-Driven Rendering & Rendering Paradigms CORE	Server-Side Rendering (SSR), Static Site Generation (SSG), Incremental Static Regeneration (ISR), Progressive Hydration, Selective Hydration, Server Components (RSC), Edge Rendering trade-offs.
Week 9	API Architecture: REST, GraphQL & gRPC CORE	REST constraints & Richardson Maturity Model; OpenAPI/Swagger specifications; GraphQL Schema definition, AST parsing, N+1 query problem, DataLoader pattern; gRPC & Protocol Buffers for web microservices.
Week 10	State Management, Caching & Persistence ADVANCED	HTTP Caching mechanisms (<code>Cache-Control</code> , <code>ETag</code> , <code>Vary</code> , Stale-While-Revalidate); Browser Storage (IndexedDB, CacheStorage API, Service Workers, Progressive Web Apps); Global State Stores.
Week 11	Web Security Architecture & Threat Vector Mitigation ADVANCED	Same-Origin Policy (SOP), CORS specification; OWASP Top 10 mitigations: Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), SQL Injection; Content Security Policy (CSP), HTTP Strict Transport Security (HSTS).
Week 12	Authentication, Authorization & Session Mechanics CORE	Session IDs & HTTP Cookies (<code>SameSite</code> , <code>HttpOnly</code> , <code>Secure</code>); Token-based auth (JWT payload signing & validation); OAuth 2.0 authorization framework & OpenID Connect (OIDC) flows.
Week 13	Real-Time Web Systems & Event-Driven Topologies APPLIED	Real-Time Architectures: WebSockets protocol framing & handshake, Server-Sent Events (SSE), WebRTC peer-to-peer data channels; Event-Driven Backends (Redis Pub/Sub, Webhook architectures).
Week 14	Web Performance Engineering & Telemetry APPLIED	Core Web Vitals (LCP, INP, CLS) measurement and optimization; Performance Profiling; Code Splitting, Tree Shaking, Dynamic Imports; Compression algorithms (Gzip, Brotli); Image/Resource optimization.
Week 15	Cloud Native Web Systems & DevOps Pipeline ADVANCED	Serverless computing (AWS Lambda, Edge Functions); Containerization (Docker) for web services; Infrastructure as Code (Terraform); CI/CD pipelines, Zero-downtime deployment strategies (Blue/Green, Canary).
Week 16	Capstone Defense & Web Systems Synthesis APPLIED	Oral defense and load-testing validation of an enterprise-grade distributed web platform (benchmarked with k6/Artillery under simulated high concurrency); final written examination.

3. ASSESSMENT & GRADING FRAMEWORK

Evaluation assesses theoretical domain mastery, network protocol analysis, security modeling, and enterprise full-stack web software engineering.

Component	Weight	Description & Objectives
Engineering Labs & Micro-Projects	25%	Practical implementations: Custom HTTP Server, React Reconciliation Profiling, GraphQL DataLoader engine, OAuth2 implementation.
System Design Assignments	15%	Architectural blueprints, security audit reports, OpenAPI definitions, and protocol packet flow diagrams.
Midterm Examination	20%	Comprehensive evaluation of Weeks 1–6: network protocols, rendering engine mechanics, and asynchronous JavaScript runtimes.
Enterprise Web Capstone	15%	End-to-end distributed web application featuring edge rendering, real-time WebSockets, strict OWASP compliance, and CI/CD pipelines.
Final Comprehensive Exam	25%	Full-spectrum exam testing web security, caching hierarchies, rendering strategies, database integration, and cloud-native architecture.

4. RECOMMENDED LITERATURE & REFERENCE MATERIALS

- Grigorik, I. *High Performance Browser Networking*. O'Reilly Media.
- Flanagan, D. *JavaScript: The Definitive Guide* (7th Ed.). O'Reilly Media.
- Banks, A., & Porcello, E. *Learning React: Modern Patterns for Developing React Apps* (2nd Ed.). O'Reilly Media.
- Stuttard, D., & Pinto, M. *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws* (2nd Ed.). Wiley.